



ELTE
FACULTY OF
INFORMATICS

PROGRAMMING

Lecture 5

Zsuzsa Pluhár
pluharzs@inf.elte.hu

Refreshing



Programming patterns

1. Summation
2. Counting
3. Maximum selection
4. Copy
5. Multiple item selection
6. Search
7. Decision
8. Selection

problems with sum and "all"
↓
counting loop

problem with "exists"
↓
conditional loop



Special cases of programming patterns



Minimum selection

Let a non-empty **interval** of integers $[b..e]$ and a **function** $f: [b..e] \rightarrow H$ be given. Let us suppose that a total order is defined on the elements of H .

Determine the **point (index)** at which attains its **MINIMUM** value on the interval $[b..e]$ and calculate this **value** too. (if there is more than one maximum, any of them can be selected)

Specification:

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{minind} \in \mathbb{Z}, \text{minval} \in H$

Pre: $b \leq e$

Post: $\text{minind} \in [b..e]$ and
 $\forall i \in [b..e]: (f(\text{minind}) \leq f(i))$ and
 $\text{minval} = f(\text{minind})$

besides the „names“, this
is the only difference

shorting:

Post:

$(\text{minind}, \text{minval}) = \text{MIN}(i = b..e, f(i))$



Minimum selection algorithm – analogous algorithmic thinking

Algorithm:

using another relational operator

<code>maxval := f(b) ; maxind := b</code>	
<code>i = (b+1) .. e</code>	
$f(i) > f(\text{maxind})$ T F	
<code>maxval := f(i)</code>	-
<code>maxind := i</code>	



<code>minval := f(b) ; minind := b</code>	
<code>i = (b+1) .. e</code>	
$f(i) < f(\text{minind})$ T F	
<code>minval := f(i)</code>	-
<code>minind := i</code>	

Minimum selection algorithm – analogous reduction

Algorithm:

- idea: transform the task:
- try to find the maximum besides the **opposites** of the function values
- the maximum we find will be the **opposite** of the minimum we are looking for

Maximum selection:

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{maxind} \in \mathbb{Z}, \text{maxval} \in H$

Pre: $b \leq e$

Post:

$(\text{maxind}, \text{maxval}) = \text{MAX}(i=b..e, f(i))$

Minimum selection:

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{minind} \in \mathbb{Z}, \text{minval} \in H$

Pre: $b \leq e$

Post:

$(\text{minind}, -\text{minval}) = \text{MIN}(i=b..e, -f(i))$

Reduction:

$\text{maxind}, \text{maxval}$	$\sim$	$\text{minind}, -\text{minval}$
$f(i)$	$\sim$	$-f(i)$

Minimum selection algorithm – analogous reduction

maxind, maxval	~	minind, -minval
f(i)	~	-f(i)

Algorithm:

maximum selection pattern

maxval := f(b) ; maxind := b	
i = (b+1) .. e	
T	f(i) > f(maxind)
	F
maxval := f(i)	-
maxind := i	

algorithmic thinking

minval := f(b) ; minind := b	
i = (b+1) .. e	
T	f(i) < f(minind)
	F
minval := f(i)	-
minind := i	

not allowed value assignment

-minval := -f(b) ; minind := b	
i = (b+1) .. e	
T	-f(i) > -f(minind)
	F
-minval := -f(i)	-
minind := i	



multiple by -1 in "not allowed" places

minval := f(b) ; minind := b	
i = (b+1) .. e	
T	f(i) < f(minind)
	F
minval := f(i)	-
minind := i	



Minimum selection – the pattern

Let a non-empty **interval** of integers $[b..e]$ and a **function** $f: [b..e] \rightarrow H$ be given.
Let us suppose that a total order is defined on the elements of H .

Determine the **point (index)** at which attains its minimum value on the interval $[b..e]$ and calculate this **value** too. (if there is more than one maximum, any of them can be selected)

Specification:

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{minind} \in \mathbb{Z}, \text{minval} \in H$

Pre: $b \leq e$

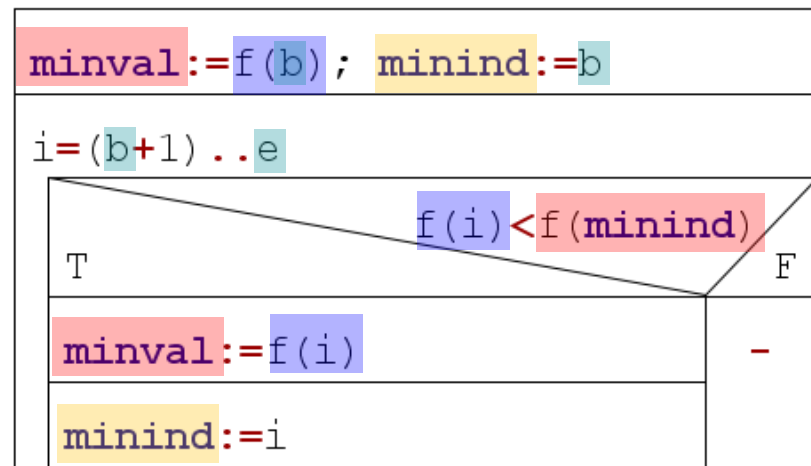
Post: $\text{minind} \in [b..e]$ and
 $\forall i \in [b..e]: (f(\text{minind}) \leq f(i))$ and
 $\text{minval} = f(\text{minind})$

shorting:

Post:

$(\text{minind}, \text{minval}) = \text{MIN}(i = b..e, f(i))$

Algorithm:



Conditional maximum selection

Let a non-empty **interval** of integers $[b..e]$ and a **function** $f: [b..e] \rightarrow H$, and a **condition** $A: [b..e] \rightarrow L$ be given. Let us suppose that a total order is defined on the elements of H .

Determine the **point (index)** at which attains its maximum value on the interval $[b..e]$ and calculate this **value** too, among the points that satisfy the **A condition**.

Specification:

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{exists} \in L, \text{maxind} \in \mathbb{Z}, \text{maxval} \in H$

Pre: -

Post: $\text{exists} = \exists i \in [b..e]: (A(i))$ and

$\text{exists} \rightarrow (\text{maxind} \in [b..e] \text{ and } \text{maxval} = f(\text{maxind})) \text{ and } A(\text{maxind})$

and $\forall i \in [b..e]: (A(i) \rightarrow f(\text{maxind}) \geq f(i))$



Conditional maximum selection

Let a non-empty **interval** of integers $[b..e]$ and a **function** $f: [b..e] \rightarrow H$, and a **condition** $A: [b..e] \rightarrow L$ be given. Let us suppose that a total order is defined on the elements of H .

Determine the **point (index)** at which attains its maximum value on the interval $[b..e]$ and calculate this **value** too, among the points that satisfy the **A condition**.

Specification:

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{exists} \in L, \text{maxind} \in \mathbb{Z}, \text{maxval} \in H$

Pre: -

Post: $(\text{exists}, \text{maxind}, \text{maxval}) = \text{CONDMAX}(i=b..e, f(i), A(i))$

Conditional maximum selection

Algorithm:

<code>exists:=false</code>			
<code>i=b..e</code>			
		exists	
T			F
		A(i) and f(i)>f(maxind)	A(i)
T			F
<code>maxval:=f(i)</code>		-	<code>maxval:=f(i)</code>
<code>maxind:=i</code>			<code>maxind:=i</code>
			<code>exists:=true</code>

Conditional maximum selection

Algorithm:

Why/how does it work with reduction?

idea:

- find the first item which attains the A condition => *selection*
- *maximum selection* from the next item with the A condition
- with program transformation (see later) -> conditional maximum selection



Conditional minimum selection

Let a non-empty **interval** of integers $[b..e]$ and a **function** $f: [b..e] \rightarrow H$, and a **condition** $A: [b..e] \rightarrow L$ be given. Let us suppose that a total order is defined on the elements of H .

Determine the **point (index)** at which attains its minimum value on the interval $[b..e]$ and calculate this **value** too, among the points that **satisfy the A condition**.

Specification:

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{exists} \in L, \text{minind} \in \mathbb{Z}, \text{minval} \in H$

Pre: -

Post: $(\text{exists}, \text{minind}, \text{minval}) = \text{CONDMIN}(i=b..e, f(i), A(i))$



Conditional minimum selection

Algorithm:

<code>exists:=false</code>			
<code>i=b..e</code>			
		exists	
T			F
	A(i) and f(i) < f(minind)		A(i)
T			F
<code>minval:=f(i)</code>		-	<code>minval:=f(i)</code>
<code>minind:=i</code>			<code>minind:=i</code>
			<code>exists:=true</code>

Decision all

Let an **interval** of integers $[b..e]$ and a **condition** $A: [b..e] \rightarrow L$ be given.

Determine whether A attains the "true" value on the interval $[b..e]$ at **all** points.

Specification:

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $all \in L$

Pre: -

Post: $all = \forall i \in [b..e]: A(i)$

decision: $\exists i \in [b..e]: (A(i))$

shorting:

Post: $all = \text{FORALL}(i=b..e, A(i))$



Decision all

algorithm – analogous algorithmic thinking

Algorithm:

step to the next item until it is „correct”. If we leave the interval, all items were „correct” (correct=satisfy the A attribute)

<code>i:=b</code>
<code>i<=e and not A(i)</code>
<code> i:=i+1</code>
<code>exists:=(i<=e)</code>

<code>i:=b</code>
<code>i<=e and A(i)</code>
<code> i:=i+1</code>
<code>all:=(i>e)</code>

Decision all algorithm – analogous reduction

Algorithm:

idea: modify (transform) the original problem

- **Is there any** item that **doesn't** have the **A attribute**?

Is there any...?

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{exists} \in \mathbb{L}$

Pre: -

Post: $\text{exists} = \text{EXISTS}(i=b..e, A(i))$

Are all...?

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{all} \in \mathbb{L}$

Pre: -

Post: $\text{all} = \text{not EXISTS}(i=b..e, A(i))$

Post: $\text{not all} = \text{EXISTS}(i=b..e, \text{not } A(i))$

Reduction:

exists	$\sim$	not all
$A(i)$	$\sim$	$\text{not } A(i)$

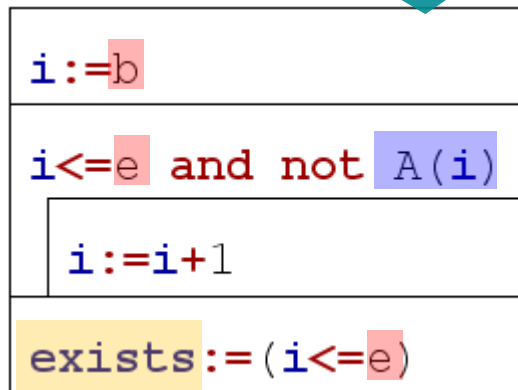


Decision all algorithm – analogous reduction

exists	~	not all
A(i)	~	not A(i)

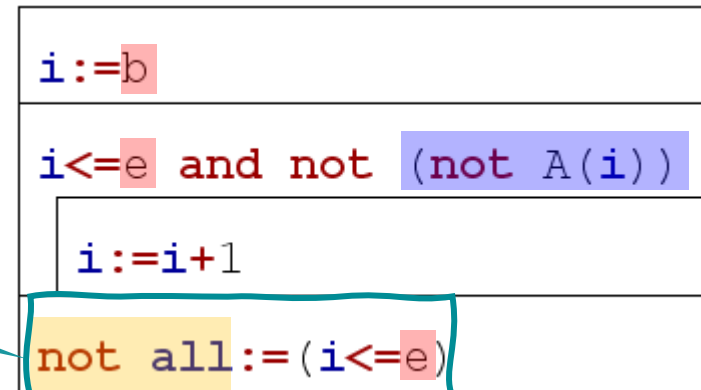
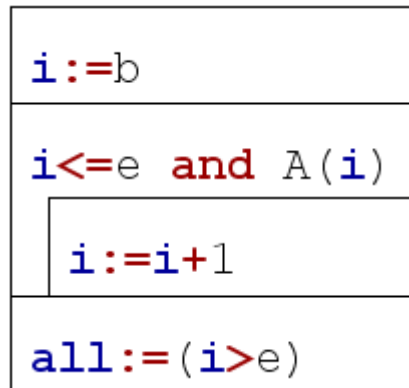
Algorithm:

decision pattern

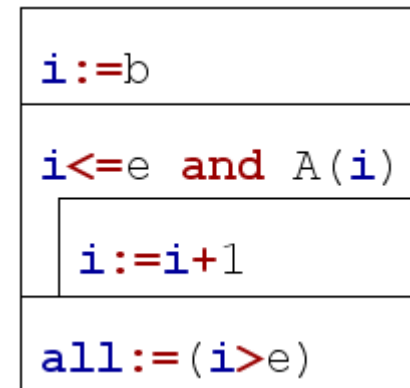


not allowed
value assignment

algorithmic thinking



negation where it is needed



Decision all - pattern

Let an **interval** of integers $[b..e]$ and a **condition** $A: [b..e] \rightarrow L$ be given.

Determine whether A attains the "true" value on the interval $[b..e]$ at **all** points.

Specification:

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $all \in L$

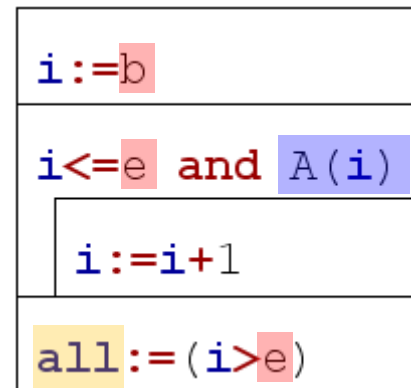
Pre: -

Post: $all = \forall i \in [b..e]: A(i)$

shorting:

Post: $all = \text{FORALL}(i=b..e, A(i))$

Algorithm:



Search – from the back

Problem: let's find the **last** item with the given A attribute.

Specification:

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{exists} \in \mathbb{L}, \text{ind} \in \mathbb{Z}$

Pre: -

Post: $\text{exists} = \exists i \in [b..e] : (A(i))$ and
 $\text{exists} \rightarrow (\text{ind} \in [b..e] \text{ and } A(\text{ind}) \text{ and } \forall i \in [\text{ind}+1..e] : (\text{not } A(i)))$

see: search – [b..ind-1]

Search – from the back

algorithm – analogous algorithmic thinking

Algorithm: let's start from the last item and step/go forward until we are in the interval and don't find an item with A attribute

<code>ind:=b</code>
<code>ind<=e and not A(ind)</code>
<code>ind:=ind+1</code>
<code>exists:=(ind<=e)</code>

<code>ind:=e</code>
<code>ind>=b and not A(ind)</code>
<code>ind:=ind-1</code>
<code>exists:=(ind>=b)</code>

Search – from the back

algorithm – analogous reduction

Algorithm:

idea: modify (transform) the original problem

- mirror the interval $[b..e]$ to the origin and find the first element with a given attribute in the „new” interval $[-b..-e]$
- the result will be $-ind$ instead of the ind

Search

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $exists \in \mathbb{L}, ind \in \mathbb{Z}$

Pre: -

Post:

$(exists, ind) = SEARCH(i=b..e, A(i))$

Search from the back

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $exists \in \mathbb{L}, ind \in \mathbb{Z}$

Pre: -

Post:

$(exists, -ind) = SEARCH(i=-b..-e, A(i))$

Reduction:

ind	$\sim$	$-ind$
$b..e$	$\sim$	$-b..-e$

Search – from the back algorithm – analogous algorithm

Algorithm:

search pattern

```
ind := b
ind <= e and not A(ind)
  ind := ind + 1
exists := (ind <= e)
```

algorithmic thinking

```
ind := e
ind >= b and not A(ind)
  ind := ind - 1
exists := (ind >= b)
```

not allowed value assignment

$b..e \sim -b..-e$

```
ind := -e
ind <= -b and not A(-ind)
  ind := ind + 1
exists := (ind <= -b)
```

$ind \sim -ind$

```
-ind := -e
-ind <= -b and not A(ind)
  -ind := -ind + 1
exists := (-ind <= -e)
```

multiple by -1 in "not allowed" places

```
ind := e
ind >= b and not A(ind)
  ind := ind - 1
exists := (ind >= b)
```


Special cases in reduction



Problem

So far, we have compared the **postcondition**, although the **programming pattern template** says something about specific data and functions interpreted on that data.

What if the data in the sample task (pattern) and the specific task are different, e.g. missing or different data?

In that case, why (and how) we are allowed to use the pattern and the shorter form of the postcondition?



Natural reduction

- Task: how many even number can be found in the interval $a..b$

Pattern (counting)

Specific task

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$



In: $a \in \mathbb{Z}, b \in \mathbb{Z}$

Out: $cnt \in \mathbb{N}$



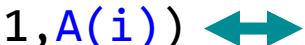
Out: $pcnt \in \mathbb{N}$

Pre: -



Pre: -

Post: $cnt = \text{SUM}(i=b..e, 1, A(i))$



Post: $pcnt = \text{SUM}(i=a..b, 1, i \bmod 2 = 0)$



Post: $cnt = \text{COUNT}(i=b..e, A(i))$

everything is perfect
only the „names” are different

Natural reduction

- Task: how many even number can be found in the interval $a..b$

Pattern (counting)

Specific task

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$



In: $a \in \mathbb{Z}, b \in \mathbb{Z}$

Out: $cnt \in \mathbb{N}$



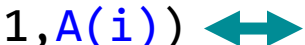
Out: $pcnt \in \mathbb{N}$

Pre: -



Pre: -

Post: $cnt = \text{SUM}(i=b..e, 1, A(i))$



Post: $pcnt = \text{SUM}(i=a..b, 1, i \bmod 2 = 0)$



Post: $cnt = \text{COUNT}(i=b..e, A(i))$



Post: $cnt = \text{COUNT}(i=a..b, i \bmod 2 = 0)$



General reduction

precondition is stricter

- Task: how many prim number can be found in interval $a..b$

Pattern (counting)

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{cnt} \in \mathbb{N}$

Pre: -

Post: $\text{cnt} = \text{SUM}(i=b..e, 1, A(i))$



Post: $\text{cnt} = \text{COUNT}(i=b..e, A(i))$

Specific task

In: $a \in \mathbb{Z}, b \in \mathbb{Z}$

Out: $\text{pcnt} \in \mathbb{N}$

Pre: $a > 0$

Post: $\text{pcnt} = \text{SUM}(i=a..b, 1, \text{prime}(i))$



Post: $\text{pcnt} = \text{COUNT}(i=a..b, \text{prime}(i))$

The specific task's precondition is stricter than the pattern. If it is correct for a set of data, it will obviously be correct for the restricted data set.

General reduction

postcondition is weaker

- Task: give a prim number between a and b

Pattern (search)

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{exists} \in \mathbb{L}, \text{ind} \in \mathbb{Z}$

Pre: -

Post: $\text{exists} = \exists i \in [b..e]: (A(i))$ and
 $\text{exists} \rightarrow (\text{ind} \in [b..e] \text{ and } A(\text{ind}))$
and $\forall i \in [b..\text{ind}-1]: (\text{not } A(i))$



Post: $(\text{exists}, \text{ind}) =$
 $\text{SEARCH}(i=b..e, A(i))$

Specific task

In: $a \in \mathbb{Z}, b \in \mathbb{Z}$

Out: $\text{ex} \in \mathbb{L}, p \in \mathbb{N}$

Pre: $a > 0$

Post: $\text{ex} = \exists i \in [a..b]: (\text{prime}(i))$ and
 $\text{ex} \rightarrow (p \in [a..b] \text{ and } \text{prime}(p))$



The specific task's postcondition is weaker than the pattern. If the problem accepts more than one solution, then obviously the narrowed (restricted) solution of the pattern will also be correct.

General reduction

postcondition is weaker

- Task: give a prim number between a and b

Pattern (search)

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{exists} \in \mathbb{L}, \text{ind} \in \mathbb{Z}$

Pre: -

Post: $\text{exists} = \exists i \in [b..e] : (A(i))$ and
 $\text{exists} \rightarrow (\text{ind} \in [b..e] \text{ and } A(\text{ind}))$
and $\forall i \in [b..\text{ind}-1] : (\text{not } A(i))$



Post: $(\text{exists}, \text{ind}) =$
 $\text{SEARCH}(i=b..e, A(i))$

Specific task

In: $a \in \mathbb{Z}, b \in \mathbb{Z}$

Out: $\text{ex} \in \mathbb{L}, p \in \mathbb{N}$

Pre: $a > 0$

Post: $\text{ex} = \exists i \in [a..b] : (\text{prime}(i))$ and
 $\text{ex} \rightarrow (p \in [a..b] \text{ and } \text{prime}(p))$



Post: $(\text{ex}, p) = \text{SEARCH}(i=a..b, \text{prime}(i))$

The specific task's postcondition is weaker than the pattern. If the problem accepts more than one solution, then obviously the narrowed (restricted) solution of the pattern will also be correct.

Sub-state-space reduction

less output

- Task: For integer values in the interval a and b , where does the function $\sin(x)$ take its highest value?

Pattern (maximum selection)

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{maxind} \in \mathbb{Z}, \text{maxval} \in \mathbb{H}$

Pre: -

Post: $\text{maxind} \in [b..e]$ and

$\forall i \in [b..e]: (f(\text{maxind}) \geq f(i))$ and

$\text{maxval} = f(\text{maxind})$



Post: $(\text{maxind}, \text{maxval}) = \text{MAX}(i=b..e, f(i))$

Specific task

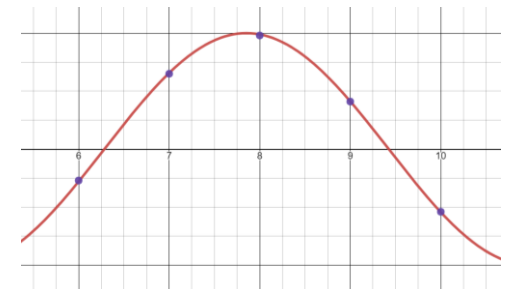
In: $a \in \mathbb{Z}, b \in \mathbb{Z}$

Out: $\text{where} \in \mathbb{Z}$

Pre: -

Post: $\text{where} \in [a..b]$ and

$\forall i \in [a..b]: (\sin(\text{where}) \geq \sin(i))$



Sub-state-space reduction less output

- Task: For integer values in the interval a and b , where does the function $\sin(x)$ take its highest value?

Pattern (maximum selection)

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{maxind} \in \mathbb{Z}$

Aux: $\text{maxval} \in \mathbb{H}$

Pre: -

Post: $\text{maxind} \in [b..e]$ and
 $\forall i \in [b..e]: (f(\text{maxind}) \geq f(i))$ and
 $\text{maxval} = f(\text{maxind})$



Post: $(\text{maxind}, \text{maxval}) = \text{MAX}(i = b..e, f(i))$

Specific task

In: $a \in \mathbb{Z}, b \in \mathbb{Z}$

Out: $\text{where} \in \mathbb{Z}$

Aux: $\text{maxval} \in \mathbb{R}$

Pre: -

Post: $\text{where} \in [a..b]$ and
 $\forall i \in [a..b]: (\sin(\text{where}) \geq \sin(i))$ and
 $\text{maxval} = \sin(\text{where})$



Post: $(\text{where}, \text{maxval}) = \text{MAX}(i = a..b, \sin(i))$

The specific task's output can be reclassified as „helper” (auxiliary) data.
→ the postcondition will be stricter (another „output data” has restriction – no problem: see „weaker postcondition”)



Sub-state-space reduction less output

- Task: For integer values in the interval a and b , where does the function $\sin(x)$ take its highest value?

Pattern (maximum selection)

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{maxind} \in \mathbb{Z}$

Aux: $\text{maxval} \in \mathbb{H}$

Pre: -

Post: $\text{maxind} \in [b..e]$ and

$\forall i \in [b..e]: (f(\text{maxind}) \geq f(i))$ and
 $\text{maxval} = f(\text{maxind})$



Post: $(\text{maxind}, \text{maxval}) = \text{MAX}(i=b..e, f(i))$

Specific task

In: $a \in \mathbb{Z}, b \in \mathbb{Z}$

Out: $\text{where} \in \mathbb{Z}$

$\text{maxind}, \text{maxval}$	$\sim$	$\text{where}, \text{maxval}$
$b..e$	$\sim$	$a..b$
$f(i)$	$\sim$	$\sin(i)$

Post: $(\text{where},) = \text{MAX}(i=a..b, \sin(i))$

The specific task doesn't need the „helper“ (auxiliary) data.

→ it can be left out of the specification altogether.

→ It is considered to have the same name in the reduction table.



Sub-state-space reduction

less output

- Task: For integer values in the interval a and b , where does

$\text{maxind}, \text{maxval}$	$\sim$	$\text{where}, \text{maxval}$
$b..e$	$\sim$	$a..b$
$f(i)$	$\sim$	$\sin(i)$

Pattern (maximum selection)

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{maxind} \in \mathbb{Z}$

Aux: $\text{maxval} \in \mathbb{H}$

Pre: -

Post: $(\text{maxind}, \text{maxval}) = \text{MAX}(i=b..e, f(i))$

$\text{maxval} := f(b); \text{maxind} := b$	
$i = (b+1) .. e$	
$f(i) > f(\text{maxind})$ T F	
$\text{maxval} := f(i)$	-
$\text{maxind} := i$	

Specific task

In: $a \in \mathbb{Z}, b \in \mathbb{Z}$

Out: $\text{where} \in \mathbb{Z}$

Pre: -

Post: $(\text{where},) = \text{MAX}(i=a..b, \sin(i))$

$\text{maxval} := \sin(a); \text{where} := a$	
$i = a+1 .. b$	
$\sin(i) > \sin(\text{where})$ T F	
$\text{maxval} := \sin(i)$	-
$\text{where} := i$	

Sub-state-space reduction

less output

- Task: For integer values in the interval a and b , where does

$\text{maxind}, \text{maxval}$	$\sim$	$\text{where}, \text{maxval}$
$b..e$	$\sim$	$a..b$
$f(i)$	$\sim$	$\sin(i)$

Pattern (maximum selection)

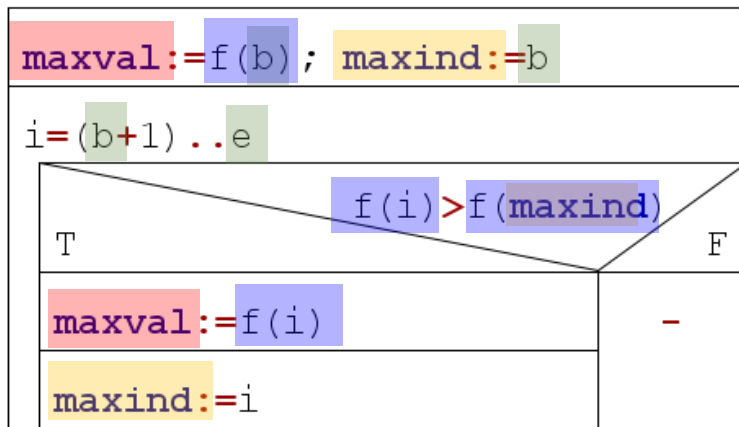
In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{maxind} \in \mathbb{Z}$

Aux: $\text{maxval} \in \mathbb{H}$

Pre: -

Post: $(\text{maxind}, \text{maxval}) = \text{MAX}(i=b..e, f(i))$



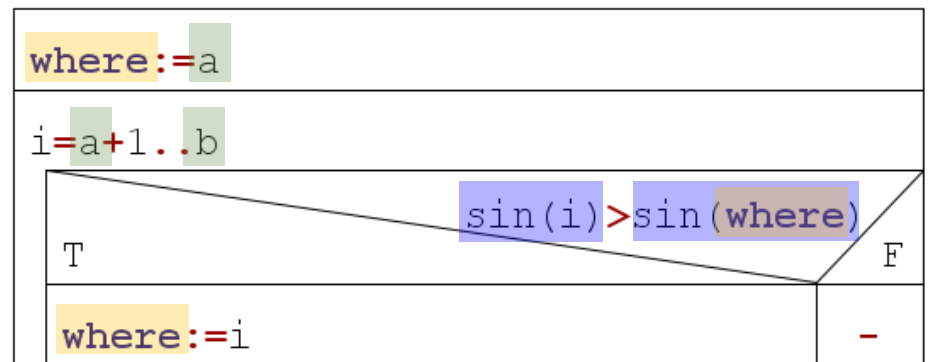
Specific task

In: $a \in \mathbb{Z}, b \in \mathbb{Z}$

Out: $\text{where} \in \mathbb{Z}$

Pre: -

Post: $(\text{where},) = \text{MAX}(i=a..b, \sin(i))$



Sub-state-space reduction

DECISION

Pattern - SEARCH

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{exists} \in \mathbb{L}, \text{ind} \in \mathbb{Z}$

Aux: $\text{ind} \in \mathbb{Z}$ 

Pre: -

Post: $(\text{exists}, \text{ind}) = \text{SEARCH}(i = b..e, A(i))$

$\text{ind} := b$

$\text{ind} \leq e$ and not $A(\text{ind})$

$\text{ind} := \text{ind} + 1$

$\text{exists} := (\text{ind} \leq e)$

Pattern - SEARCH

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{exists} \in \mathbb{L}$

Aux: $\text{ind} \in \mathbb{Z}$

Pre: -

Post: $(\text{exists}, \text{ind}) = \text{SEARCH}(i = b..e, A(i))$

Post: $(\text{exists},) = \text{SEARCH}(i = b..e, A(i))$

Post: $\text{exists} = \text{EXISTS}(i = b..e, A(i))$

$i := b$

$i \leq e$ and not $A(i)$

$i := i + 1$

$\text{exists} := (i \leq e)$

Sub-state-space reduction

less input

- Task: Let's calculate the number of divisors of a natural number

Pattern (counting)

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $cnt \in \mathbb{N}$

Pre: -

Post: $cnt = \text{SUM}(i=b..e, 1, A(i))$



Post: $cnt = \text{COUNT}(i=b..e, A(i))$

Specific task

In: $n \in \mathbb{N}$

Out: $cnt \in \mathbb{N}$

Pre: -

Post: $cnt = \text{SUM}(i=2..n, 1, i|n)$

If an input data is missing that doesn't change the value $\rightarrow$ can be replaced with a *constant*. $\rightarrow$ the problem (task) will not change.

We can define it in the precondition – the precondition of the specific task can be stricter than the pattern's precondition

Sub-state-space reduction

less input

- Task: Let's calculate the number of divisors of a natural number

Pattern (counting)

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{cnt} \in \mathbb{N}$

Pre: -

Post: $\text{cnt} = \text{SUM}(i=b..e, 1, A(i))$



Post: $\text{cnt} = \text{COUNT}(i=b..e, A(i))$

Specific task

In: $b \in \mathbb{N}, n \in \mathbb{N}$

Out: $\text{cnt} \in \mathbb{N}$

Pre: $b=2$

Post: $\text{cnt} = \text{SUM}(i=2..n, 1, i|n)$



Post: $\text{cnt} = \text{COUNT}(i=b..n, i|n)$

If an input data is missing that doesn't change the value → can be replaced with a *constant*. → the problem (task) will not change.

We can define it in the precondition – the precondition of the specific task can be stricter than the pattern's precondition

Parametric reduction

more input

- Task: Does k divide at least one number between a and b ?

Pattern (decision)

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{exists} \in \mathbb{L}$

Pre: -

Post: $\text{exists} = \exists i \in [b..e] : (A(i))$



Post: $\text{exists} = \text{EXISTS}(i=b..e, A(i))$

Specific task

In: $a \in \mathbb{N}, b \in \mathbb{N}, k \in \mathbb{N}$

Out: $k \text{div} \in \mathbb{L}$

Pre: $k > 0$

Post: $k \text{div} = \exists i \in [a..b] : (k | i)$

If the „extra” input data doesn’t change the value $\rightarrow$ can be replaced with a *constant*. $\rightarrow$ the problem (task) will not change, the reduction can be done.
This will work with all value of k .



Parametric reduction

more input

- Task: Does k divide at least one number between a and b ?

Pattern (decision)

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{exists} \in \mathbb{L}$

Pre: -

Post: $\text{exists} = \exists i \in [b..e] : (A(i))$



Post: $\text{exists} = \text{EXISTS}(i=b..e, A(i))$

Specific task

In: $a \in \mathbb{N}, b \in \mathbb{N}, k \in \mathbb{N}$

Out: $k \text{div} \in \mathbb{L}$

Pre: $k > 0$

Post: $k \text{div} = \exists i \in [a..b] : (k | i)$



Post: $k \text{div} = \text{EXISTS}(i=a..b, k | i)$

for all k

If the „extra” input data doesn’t change the value $\rightarrow$ can be replaced with a *constant*. $\rightarrow$ the problem (task) will not change, the reduction can be done.
This will work with all value of k .



Example

they don't look like similar – why can we do the reduction?

- Task: Determine the highest temperature...

Pattern (maximum selection)

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{maxind} \in \mathbb{Z}, \text{maxval} \in \mathbb{H}$

Pre: $b \leq e$

Post: $(\text{maxind}, \text{maxval}) =$
 $\text{MAX}(i=b..e, f(i))$

$\text{maxval} := f(b); \text{maxind} := b$	
$i := (b+1) .. e$	
$f(i) > f(\text{maxind})$	
T	F
$\text{maxval} := f(i)$	-
$\text{maxind} := i$	

$\text{maxind}, \text{maxval}$	$\sim$	$\text{maxind}, \text{http}$
$b..e$	$\sim$	$1..n$
$f(i)$	$\sim$	$\text{temp}[i]$

Specific task

In: $n \in \mathbb{N}, \text{temp} \in \mathbb{R}[1..n]$

Out: $\text{http} \in \mathbb{R}$

Pre: $n > 0$ and

$\forall i \in [1..n]: (-100 \leq \text{temp}[i] \leq 100)$

Post: $(, \text{http}) =$
 $\text{MAX}(i=1..n, \text{temp}(i))$

$\text{http} := \text{temp}[1]$	
$i := 2 .. n$	
$\text{temp}[i] > \text{http}$	
T	F
$\text{http} := \text{temp}[i]$	-



Example

- Task: Determine the highest temperature...

Specific task

In: $n \in \mathbb{N}$, $\text{temp} \in \mathbb{R}[1..n]$

Out: $\text{htmp} \in \mathbb{R}$

Pre: $n > 0$ and

$\forall i \in [1..n]: (-100 \leq \text{temp}[i] \leq 100)$

Post: $\exists i \in [1..n]: (\text{htmp} = \text{temp}[i])$ and

$\forall i \in [1..n]: \text{htmp} \geq \text{temp}[i]$



it looks like a maximum selection

Example

- Task: Determine the highest temperature...

Pattern (maximum selection)

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{maxind} \in \mathbb{Z}, \text{maxval} \in H$

Pre: $b \leq e$

Post: $\text{maxind} \in [b..e]$ and
 $\forall i \in [b..e]: (f(\text{maxind}) \geq f(i))$ and
 $\text{maxval} = f(\text{maxind})$



Post: $(\text{maxind}, \text{maxval}) =$
 $\text{MAX}(i = b..e, f(i))$

Specific task

In: $n \in \mathbb{N}, \text{temp} \in \mathbb{R}[1..n]$

Out: $\text{htmp} \in \mathbb{R}$

Pre: $n > 0$ and
 $\forall i \in [1..n]: (-100 \leq \text{temp}[i] \leq 100)$

Post: $\exists i \in [1..n]: (\text{htmp} = \text{temp}[i])$ and
 $\forall i \in [1..n]: \text{htmp} \geq \text{temp}[i]$

it looks like a maximum selection – but the similarity is poor

Example

- Task: Determine the highest temperature...

Pattern (maximum selection)

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{maxind} \in \mathbb{Z}, \text{maxval} \in H$

Pre: $b \leq e$

Post: $\text{maxind} \in [b..e]$ and
 $\forall i \in [b..e]: (f(\text{maxind}) \geq f(i))$ and
 $\text{maxval} = f(\text{maxind})$



Post: $(\text{maxind}, \text{maxval}) =$
 $\text{MAX}(i = b..e, f(i))$

Specific task

In: $n \in \mathbb{N}, \text{temp} \in \mathbb{R}[1..n]$

Out: $\text{htmp} \in \mathbb{R}$

Pre: $n > 0$ and
 $\forall i \in [1..n]: (-100 \leq \text{temp}[i] \leq 100)$

Post: $\exists i \in [1..n]: (\text{htmp} = \text{temp}[i])$ and
 $\forall i \in [1..n]: \text{htmp} \geq \text{temp}[i]$

1. Parametric reduction
the value of the items in array will be not changed

Example

- Task: Determine the highest temperature...

Pattern (maximum selection)

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{maxind} \in \mathbb{Z}, \text{maxval} \in \mathbb{H}$

Pre: $b \leq e$

Post: $\text{maxind} \in [b..e]$ and
 $\forall i \in [b..e]: (f(\text{maxind}) \geq f(i))$ and
 $\text{maxval} = f(\text{maxind})$ 

Post: $(\text{maxind}, \text{maxval}) =$
 $\text{MAX}(i = b..e, f(i))$

Specific task

In: $b \in \mathbb{N}, e \in \mathbb{N}, n \in \mathbb{N}, \text{temp} \in \mathbb{R}[1..n]$

Out: $\text{htmp} \in \mathbb{R}$

Pre: $n > 0$ and

$\forall i \in [1..n]: (-100 \leq \text{temp}[i] \leq 100)$

and $b = 1$ and $e = n$ and $b \leq e$

Post: $\exists i \in [1..n]: (\text{htmp} = \text{temp}[i])$ and
 $\forall i \in [1..n]: \text{htmp} \geq \text{temp}[i]$

2. Sub-state-space reduction (less input)
the endpoints of the interval are explicit included.
They are constants $\rightarrow$ we can write them in Input
and define in the precondition.

Example

- Task: Determine the highest temperature...

Pattern (maximum selection)

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{maxind} \in \mathbb{Z}, \text{maxval} \in \mathbb{H}$

Aux: $\text{maxind} \in \mathbb{Z}$

Pre: $b \leq e$

Post: $\text{maxind} \in [b..e]$ and
 $\forall i \in [b..e]: (f(\text{maxind}) \geq f(i))$ and
 $\text{maxval} = f(\text{maxind})$

Post: $(\text{maxind}, \text{maxval}) =$
 $\text{MAX}(i = b..e, f(i))$

Specific task

In: $b \in \mathbb{N}, e \in \mathbb{N}, n \in \mathbb{N}, \text{temp} \in \mathbb{R}[1..n]$

Out: $\text{htmp} \in \mathbb{R}$

Aux: $\text{maxind} \in \mathbb{Z}$

Pre: $n > 0$ and
 $\forall i \in [1..n]: (-100 \leq \text{temp}[i] \leq 100)$
and $b = 1$ and $e = n$ and $b \leq e$

Post: $\exists i \in [1..n]: (\text{htmp} = \text{temp}[i])$ and
 $\forall i \in [1..n]: \text{htmp} \geq \text{temp}[i]$

3. Sub-state-space reduction (less output)
the pattern's output can be changed to „helper”
data...

Example

- Task: Determine the highest temperature...

Pattern (maximum selection)

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{maxval} \in \mathbb{H}$

Aux: $\text{maxind} \in \mathbb{Z}$

Pre: $b \leq e$

Post: $\text{maxind} \in [b..e]$ and
 $\forall i \in [b..e]: (f(\text{maxind}) \geq f(i))$ and
 $\text{maxval} = f(\text{maxind})$

Post: $(\text{maxind}, \text{maxval}) =$
 $\text{MAX}(i = b..e, f(i))$



Specific task

In: $b \in \mathbb{N}, e \in \mathbb{N}, n \in \mathbb{N}, \text{temp} \in \mathbb{R}[1..n]$

Out: $\text{htmp} \in \mathbb{R}$

Aux: $\text{maxind} \in \mathbb{Z}$

Pre: $n > 0$ and
 $\forall i \in [1..n]: (-100 \leq \text{temp}[i] \leq 100)$
and $b = 1$ and $e = n$ and $b \leq e$

Post: $\exists i \in [1..n]: (\text{htmp} = \text{temp}[i])$ and
 $\forall i \in [1..n]: \text{htmp} \geq \text{temp}[i]$

4. General reduction (stricter precondition)
the precondition is stricter – it narrows the domain...

Example

- Task: Determine the highest temperature...

Pattern (maximum selection)

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{maxval} \in \mathbb{H}$

Aux: $\text{maxind} \in \mathbb{Z}$

Pre: $b \leq e$

Post: $\text{maxind} \in [b..e]$ and
 $\forall i \in [b..e]: (f(\text{maxind}) \geq f(i))$ and
 $\text{maxval} = f(\text{maxind})$

Post: $(\text{maxind}, \text{maxval}) =$
 $\text{MAX}(i = b..e, f(i))$

Specific task

In: $b \in \mathbb{N}, e \in \mathbb{N}, n \in \mathbb{N}, \text{temp} \in \mathbb{R}[1..n]$

Out: $\text{htmp} \in \mathbb{R}$

Aux: $\text{maxind} \in \mathbb{Z}$

Pre: $n > 0$ and
 $\forall i \in [1..n]: (-100 \leq \text{temp}[i] \leq 100)$
and $b = 1$ and $e = n$ and $b \leq e$

Post: $\text{maxind} \in [1..n]$ and
 $\forall i \in [1..n]: \text{temp}[\text{maxind}] \geq \text{temp}[i]$
and $\text{htmp} = \text{temp}[\text{maxind}]$

4. General reduction (weaker postcondition)

the postcondition can be modified: what do we know about htmp, b, e and temp → we create a stricter postcondition

Example

- Task: Determine the highest temperature...

Pattern (maximum selection)

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{maxval} \in \mathbb{H}$

Aux: $\text{maxind} \in \mathbb{Z}$

Pre: $b \leq e$

Post: $\text{maxind} \in [b..e]$ and
 $\forall i \in [b..e]: (f(\text{maxind}) \geq f(i))$ and
 $\text{maxval} = f(\text{maxind})$

Post: $(\text{maxind}, \text{maxval}) =$
 $\text{MAX}(i = b..e, f(i))$

Specific task

In: $b \in \mathbb{N}, e \in \mathbb{N}, n \in \mathbb{N}, \text{temp} \in \mathbb{R}[1..n]$

Out: $\text{htmp} \in \mathbb{R}$

Aux: $\text{maxind} \in \mathbb{Z}$

Pre: $b \leq e$ and $b = 1$ and $e = n$ and
 $n > 0$ and

$\forall i \in [1..n]: (-100 \leq \text{temp}[i] \leq 100)$

Post: $\text{maxind} \in [1..n]$ and
 $\forall i \in [1..n]: \text{temp}[\text{maxind}] \geq \text{temp}[i]$
and $\text{htmp} = \text{temp}[\text{maxind}]$

reorder the data/pre- and postcondition → we have the pattern

Example

the reduction can be done

$\text{maxind}, \text{maxval}$	$\sim$	$\text{maxind}, \text{htmp}$
$b..e$	$\sim$	$1..n$
$f(i)$	$\sim$	$\text{temp}[i]$

- Task: Determine the highest temperature...

Pattern (maximum selection)

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{maxval} \in \mathbb{H}$

Aux: $\text{maxind} \in \mathbb{Z}$

Pre: $b \leq e$

Post: $\text{maxind} \in [b..e]$ and
 $\forall i \in [b..e]: (f(\text{maxind}) \geq f(i))$ and
 $\text{maxval} = f(\text{maxind})$



Post: $(\text{maxind}, \text{maxval}) =$
 $\text{MAX}(i = b..e, f(i))$

Specific task

In: $b \in \mathbb{N}, e \in \mathbb{N}, n \in \mathbb{N}, \text{temp} \in \mathbb{R}[1..n]$

Out: $\text{htmp} \in \mathbb{R}$

Aux: $\text{maxind} \in \mathbb{Z}$

Pre: $b \leq e$ and $b = 1$ and $e = n$ and
 $n > 0$ and
 $\forall i \in [1..n]: (-100 \leq \text{temp}[i] \leq 100)$

Post: $\text{maxind} \in [1..n]$ and
 $\forall i \in [1..n]: \text{temp}[\text{maxind}] \geq \text{temp}[i]$
and $\text{htmp} = \text{temp}[\text{maxind}]$



Post: $(\text{maxind}, \text{htmp}) =$
 $\text{MAX}(i = 1..n, \text{temp}[i])$

Example

- Task: Determine the highest temperature...

Pattern (maximum selection)

In: $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out: $\text{maxind} \in \mathbb{Z}, \text{maxval} \in \mathbb{H}$

Pre: $b \leq e$

Post: $(\text{maxind}, \text{maxval}) =$
 $\text{MAX}(i=b..e, f(i))$

Specific task

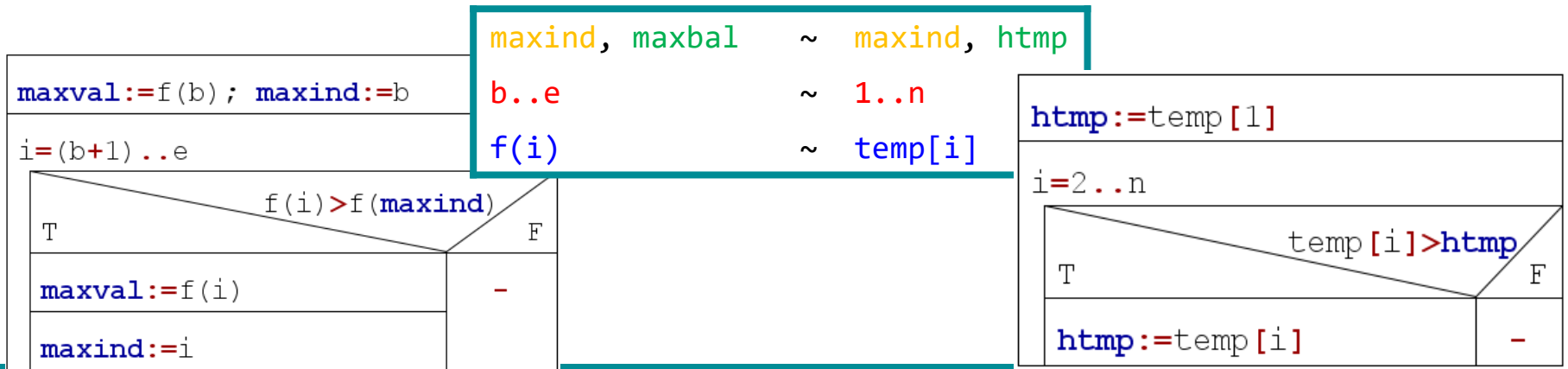
In: $n \in \mathbb{N}, \text{temp} \in \mathbb{R}[1..n]$

Out: $\text{htmp} \in \mathbb{R}$

Pre: $n > 0$ and

$\forall i \in [1..n]: (-100 \leq \text{temp}[i] \leq 100)$

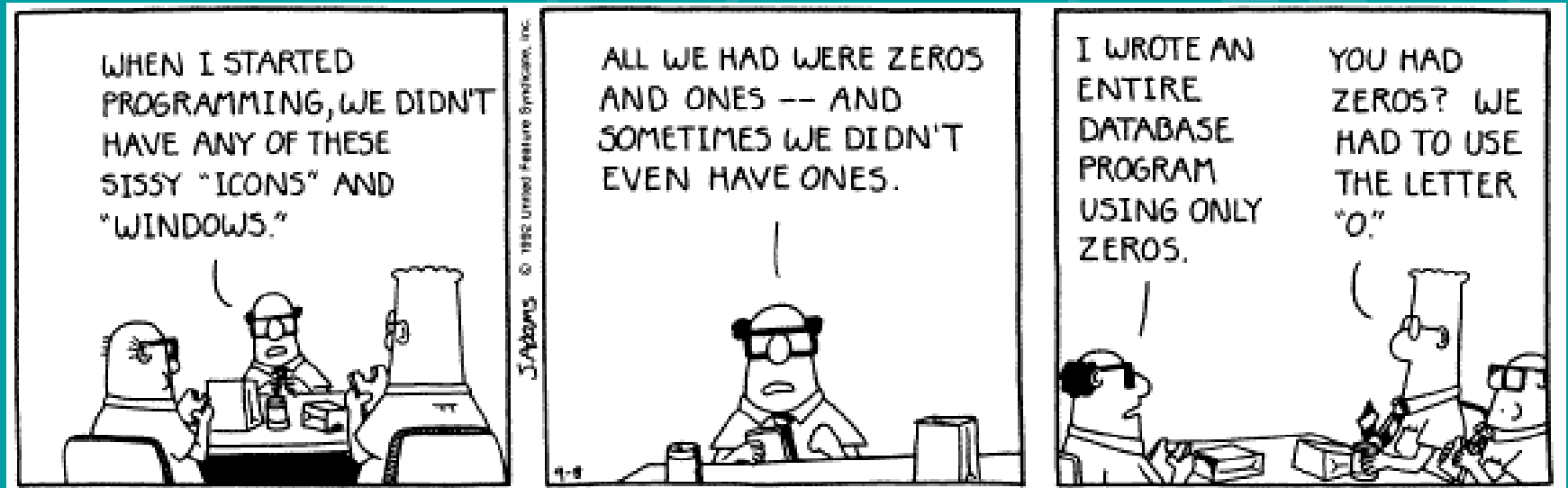
Post: $(\text{maxind}, \text{htmp}) =$
 $\text{MAX}(i=1..n, \text{temp}[i])$





ELTE

FACULTY OF
INFORMATICS



Thank you for your attention!